High Tech Marketing Text Analysis

Brand: Motorola Razr 2025

Group 12?

Introduction

Explain shit here philippe 8=====D

Sample Selection

Data Sources

Explain which sources we used to grab the data and why these are good sources for the analysis

Sampling representation & Bias

Explain if the sample is representative of all customers, and how platform choice may skew results e.g Reddit chuds are not normal people.

Data Collection

```
fetch_reddit_thread <- function(thread_url, limit = 500, attempts = 5) {
  api_url <- paste0(thread_url, ".json?limit=", limit)
  response <- RETRY(
    "GET",
    api_url,
    user_agent("Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) C
    times = attempts,
    pause_base = 2,
    pause_cap = 20,
    terminate_on = c(400, 401, 403, 404)
  )

  if (http_error(response)) {
    stop("HTTP ", status_code(response), " when fetching ", api_url)
  }

  payload <- fromJSON(content(response, as = "text", encoding = "UTF-8"), simplifyVector = FALSE)
  post <- payload[[1]]$data$children[[1]]$data
```

```r
  flatten_comments <- function(children) {
    if (!is.list(children)) {
      return(data.frame())
    }

    rows <- list()

    for (child in children) {
      if (!is.null(child$kind) && child$kind == "t1") {
        comment <- child$data
        rows[[length(rows) + 1]] <- data.frame(
          type = "comment",
          id = comment$id %||% NA_character_,
          parent_id = comment$parent_id %||% NA_character_,
          author = comment$author %||% NA_character_,
          body = comment$body %||% NA_character_,
          score = comment$score %||% NA_real_,
          created_utc = comment$created_utc %||% NA_real_,
          stringsAsFactors = FALSE
        )
        replies <- if (is.list(comment$replies) && !is.null(comment$replies$data$children)) {
          comment$replies$data$children
        } else {
          list()
        }
        if (length(replies) > 0) {
          rows <- c(rows, flatten_comments(replies))
        }
      }
    }

    if (length(rows) == 0) {
      return(data.frame())
    }

    do.call(rbind, rows)
  }

  comments <- flatten_comments(payload[[2]]$data$children %||% list())

  data.frame(
    type = c("post", comments$type),
    id = c(post$id %||% NA_character_, comments$id),
    parent_id = c(NA_character_, comments$parent_id),
    author = c(post$author %||% NA_character_, comments$author),
    body = c(post$selftext %||% post$title %||% NA_character_, comments$body),
    score = c(post$score %||% NA_real_, comments$score),
    created_utc = c(post$created_utc %||% NA_real_, comments$created_utc),
    stringsAsFactors = FALSE
  )
}

# if the cache exists and has content, use it; otherwise fetch fresh Reddit data.
```

```
reddit_cache <- "data/reddit_content.csv"
cache_ready <- file.exists(reddit_cache) && file.info(reddit_cache)$size > 1

if (cache_ready) {
  raw_df <- tryCatch(
    read.csv(reddit_cache, stringsAsFactors = FALSE),
    error = function(e) {
      message("Reddit cache could not be read: ", conditionMessage(e))
      data.frame()
    }
  )

  if (nrow(raw_df) > 0) {
    cat("Loaded Reddit content from cache:", nrow(raw_df), "\n")
  } else {
    message("Reddit cache is empty, fetching fresh data instead.")
    cache_ready <- FALSE
  }
}
```

```
## Reddit cache could not be read: first five rows are empty: giving up
```

```
## Reddit cache is empty, fetching fresh data instead.
```

```
if (!cache_ready) {
  reddit_posts <- tryCatch(
    find_thread_urls(keywords = "motorola razr 2025",
                     subreddit = "Android", sort_by = "top"),
    error = function(e) {
      message("Reddit search failed: ", conditionMessage(e))
      data.frame(url = character())
    }
  )

  cat("Reddit thread URLs found:", nrow(reddit_posts), "\n")

  raw_df <- if (nrow(reddit_posts) > 0) {
    tryCatch(
      fetch_reddit_thread(reddit_posts$url[1]),
      error = function(e) {
        message("Reddit content fetch failed: ", conditionMessage(e))
        data.frame()
      }
    )
  } else {
    message("No Reddit threads matched the search query.")
    data.frame()
  }

  # save the raw data to csv for future use and to avoid hitting api limits while developing
  write.csv(raw_df, reddit_cache, row.names = FALSE)
}
```

```
## parsing URLs on page 1...
## Reddit thread URLs found: 12

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 2

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 3

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 4

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 5

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 6

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 7

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 8

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 2

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 3

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 4

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 5

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 6

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 7

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 8

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 2

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 3
```

```
## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 4

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 5

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 6

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 7

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 8

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 9 of arg 2

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 9 of arg 3

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 9 of arg 4

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 9 of arg 5

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 9 of arg 6

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 9 of arg 7

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 9 of arg 8

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 11

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 12

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 13

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 14

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 15
```

```
## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 16

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 17

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 2

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 3

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 4

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 5

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 6

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 7

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 8

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 2

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 3

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 4

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 5

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 6

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 7

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 8

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 10
```

```
## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 11

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 12

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 13

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 14

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 15

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 16

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 3

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 4

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 5

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 6

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 7

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 8

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 8 of arg 9

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 16 of arg 11

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 16 of arg 12

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 16 of arg 13

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 16 of arg 14
```

```
## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 16 of arg 15

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 16 of arg 16

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 16 of arg 17

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 2

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 3

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 4

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 5

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 6

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 7

## Warning in rbind(deparse.level, ...): number of columns of result, 7, is not a
## multiple of vector length 17 of arg 8
```

```
cat("Rows collected:", nrow(raw_df), "\n")
```

```
## Rows collected: 20
```

```
cat("Posts vs comments:\n")
```

```
## Posts vs comments:
```

```
print(table(raw_df$type))
```

```
##
##
##
##
##
##
##
##
##
```

```
## Agreed; I can't wait for them to drop the Razr Fold 2027 next year and act like this one is obsolete
##
##
##
##
##
##
##
##
##
##
##
##
##
##
##
##
##
##
##
```

```
# we fetched via outscrapper which was exported as a a csv so we just read the file here :)
# due to limitations with the api price, only a subset of reviews are collected, from a subset of razr

#cat("Reviews collected:", nrow(amazon_reviews), "\n")
#cat("Rating distribution:\n")
#print(table(amazon_reviews$rating))
```

```
fetch_youtube_comments <- function(video_id, max_comments = 200, api_key = Sys.getenv("YOUTUBE_KEY")) {
  if (identical(api_key, "")) {
    stop("YOUTUBE_KEY is not set. Load it from .env or your environment before knitting.")
  }

  all_comments <- list()
  page_token   <- NULL

  repeat {
    query <- list(
      part       = "snippet",
      videoId    = video_id,
      maxResults = 100,
      textFormat = "plainText",
      key        = api_key
    )

    # Add pagination token if we have one
    if (!is.null(page_token)) {
      query$pageToken <- page_token
    }

    resp <- httr::GET(
      "https://www.googleapis.com/youtube/v3/commentThreads",
      query = query
    )
```

```
  if (httr::http_error(resp)) {
    error_body <- tryCatch(
      httr::content(resp, as = "text", encoding = "UTF-8"),
      error = function(e) ""
    )
    message("YouTube API error: ", httr::status_code(resp), if (nzchar(error_body)) paste0(" - ", err
    break
  }

  parsed <- jsonlite::fromJSON(
    httr::content(resp, as = "text", encoding = "UTF-8"),
    flatten = TRUE
  )

  # Extract top-level comments
  items <- parsed$items

  if (is.null(items) || nrow(items) == 0) break

  batch <- data.frame(
    comment_id    = items$id,
    author        = items$snippet.topLevelComment.snippet.authorDisplayName,
    body          = items$snippet.topLevelComment.snippet.textDisplay,
    like_count    = items$snippet.topLevelComment.snippet.likeCount,
    reply_count   = items$snippet.totalReplyCount,
    date          = items$snippet.topLevelComment.snippet.publishedAt,
    stringsAsFactors = FALSE
  )

  all_comments[[length(all_comments) + 1]] <- batch
  collected <- sum(sapply(all_comments, nrow))
  message("Collected: ", collected, " comments")

  # Check for next page
  page_token <- parsed$nextPageToken

  if (is.null(page_token) || collected >= max_comments) break

  Sys.sleep(0.5)
}

if (length(all_comments) == 0) return(data.frame())

dplyr::bind_rows(all_comments) %>%
  dplyr::mutate(
    source  = "youtube",
    # Like count as proxy for sentiment strength
    segment = dplyr::case_when(
      like_count >= quantile(like_count, 0.66, na.rm = TRUE) ~ "positive",
      like_count <= quantile(like_count, 0.33, na.rm = TRUE) ~ "negative",
      TRUE ~ "neutral"
    )
  )
```

```
}
razr_videos <- list(
  mkbhd      = "8om1eJrO2lU",    # MKBHD Razr 2025 review
  dave2d     = "JTT67FSc8j8",    # shortCircuit review
  mrmobile   = "YGTkjchlVJk",    # MrMobile review
  phonearena = "b3Ijuf7DHcQ"     # PhoneArena review
)

# to save on api calls, we check if we have a local csv of comments already, if not we fetch and save f
if (file.exists("data/youtube_comments.csv")) {
  youtube_all <- read.csv("data/youtube_comments.csv", stringsAsFactors = FALSE)
  cat("Loaded YouTube comments from CSV:", nrow(youtube_all), "\n")
} else {
  youtube_all <- purrr::map2_dfr(
    razr_videos,
    names(razr_videos),
  ~   fetch_youtube_comments(.x, max_comments = 200) %>%
        dplyr::mutate(channel = .y)
  )
  # Save to CSV for future runs
  write.csv(youtube_all, "data/youtube_comments.csv", row.names = FALSE);
}
```

```
## Loaded YouTube comments from CSV: 704
```

```
cat("YouTube comments collected:", nrow(youtube_all), "\n")
```

```
## YouTube comments collected: 704
```

```
print(table(youtube_all$channel))
```

```
##
##     dave2d      mkbhd   mrmobile phonearena
##        200        200        200        104
```

Pre-Processing

Side note maybe we dont have to do all of them based on the quality of the data we get, but we should at least explain how we would do them and why they are important. ## Filtering & Cleaning Explain how we cleaned the data and why.

Tokenization

Explain how we tokenized the data and why.

Stop Words Removal

Explain how we removed stop words and why.

Stemming & Lemmatization

Frequency Filter

Synonym consolidation

Summary Statistics

Topic Modeling (LDA)

Build Document

Optimal K

Fit Model

Stability

Results

Topic Word Distributions

Sometother shit

Reflection